

DARES Electronics

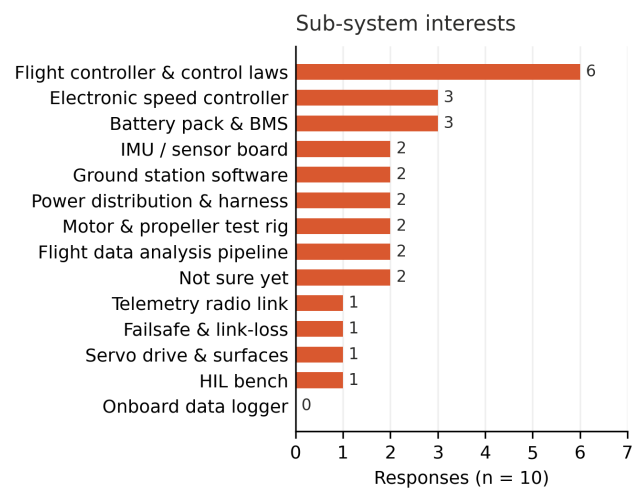
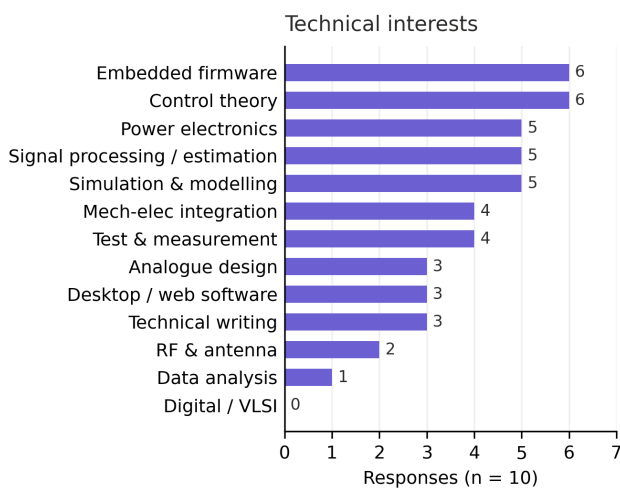
Project structure proposal

Prepared by David Shmail • 14 September 2026 • v1 for internal sign-off

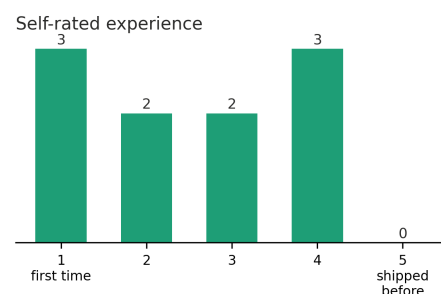
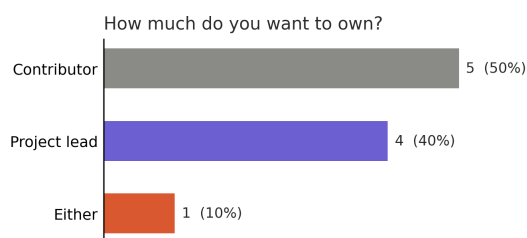
Summary

Ten people answered the onboarding form. Flight control dominates their interest (6 of 9 unique respondents), three people want to lead, and three software-oriented members have no hardware home yet. Assuming half the team commits weekly hours, we can sustain **six projects of two to three people**: the three that already exist, widened where the form asks for it, plus three new ones that give the software people a product, split the flight-control overflow, and formalise the test bench. Together they own every electrical function needed to fly, and each produces numbers the FDR depends on. Members who have not submitted the form are listed but unallocated.

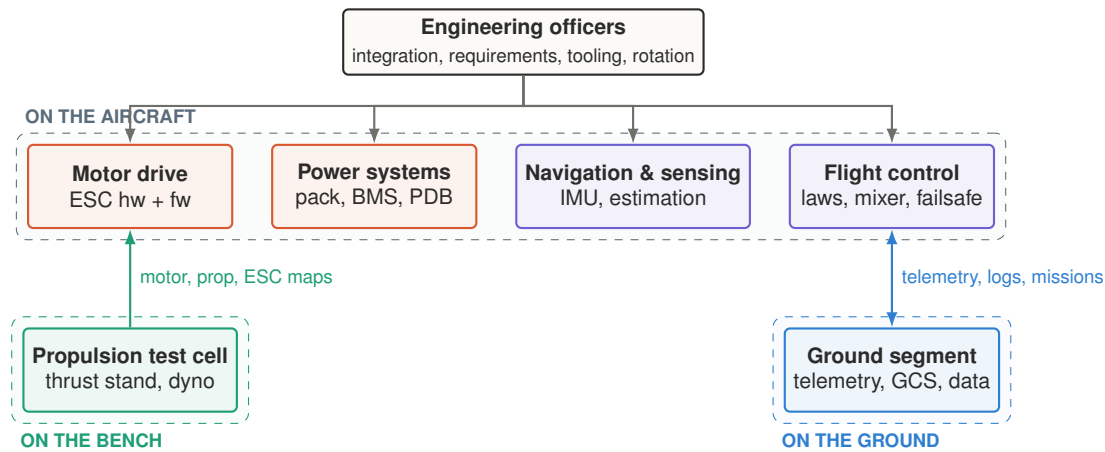
What the form told us



- **Flight control is the gravity well.** Six people chose it. One project cannot hold that many; it splits into control and estimation.
- **Three leads, three existing projects.** Rohan (flight control), Jayden (ESC), Changan (BMS). This is the backbone.
- **Three software people, no home.** Wenrui, Leah and Yifei picked data pipeline, ground station and “put me where I’m useful”.
- **People want the maths, not the board.** Signal processing scored 5, the IMU board 2. Estimation is a project; board design is a task inside it.
- **Nine of ten want the PID + IMU workshop.** That is the first workshop, not a project.
- **Scope reality.** Roughly 12 active members, half reliably present: six reliable engineers. Six projects of two is the ceiling.



Proposed structure



STAFFING FROM THE FORM

#	Project and scope	Lead	Members	Status
1	Flight control – control laws, mixer, failsafe logic, HIL	Rohan Lynch	Nicholas Tran, Alfred Wang	existing
2	Motor drive – ESC hardware and firmware, commutation, FOC	Jayden Filleul	<i>needs members</i>	existing
3	Power systems – pack, BMS, power distribution, harness, safety switch	Changhan Zhang	Alfred Wang (shared)	existing, widened
4	Navigation & sensing – IMU/sensor board, filtering, state estimation	<i>TBD</i>	Yifei Sun, Nicholas Tran (shared)	new
5	Ground segment – telemetry link, ground station, logging, data pipeline	<i>TBD</i>	Wenrui Chen, Leah Chen	new
6	Propulsion test cell – thrust stand, dyno, motor/prop/ESC characterisation	<i>TBD</i>	Zhen Rong (Drew) Fong	new; absorbs the test bench repo

Officers, floating across all projects: Jinghan Li (engineering officer) and David Shmail (systems and software, integration, tooling). Integration and requirements traceability are officer responsibilities, not a project; rotating members through them later is how we build system-level awareness.

Not yet allocated – form not submitted (13): Amelia Zhu, Arshaan Khan, Ash Zhang, Dharendra Singh, Harry Nguyen, Jiarong Chen, Joshua Pacifici, Kieren Lim, Mahin Shams, Shangrui Wu, Wen Gan, Zak Wan Ullok, Zixi Chen. They are placed the moment they submit the form; until then they attend the general meeting and workshops. Motor drive and the test cell are the first two projects newcomers should be steered toward: hands-on, least dependent on prior theory, and short of people.

The projects

Each project is described the same way: purpose, what it delivers as a black box, milestones in order (not on a timeline), a broad and adoptable tool stack, what we can hand the team next week, and a self-directed research brief that names the early architectural decisions the team is expected to make for itself.

1 Flight control

Rohan Lynch • Nicholas Tran • Alfred Wang

PURPOSE Keep the aircraft where it is told to be, recover it when things go wrong, and produce the flight behaviour the FDR predicted.

BLACK BOX *In:* state estimate, pilot or mission setpoints, link status. *Out:* motor and servo commands, flight mode, failsafe actions, logged controller states.

MILESTONES **M0** baseline platform chosen, simulator running → **M1** rate loop stable on the bench (HIL) → **M2** attitude loop and mixer, first flight on a COTS flight controller → **M3** failsafe and mission modes → **M4** predicted-vs-measured flight characterisation for the FDR.

STACK PX4 or ArduPilot as reference implementation; C/C++ on an STM32-class autopilot; MATLAB/Simulink or Python for the plant model; SITL/HIL (jMAVSim, Gazebo, Simulink) before any flight.

NEXT WEEK Pick the baseline (COTS autopilot to fly on vs. what we write ourselves); get a fixed-wing SITL running end to end; list every rule in the competition document that touches control or emergency procedures.

Self-directed research

[cascaded PID](#)
[rate loop](#)
[TECS](#)
[L1 / NPFG guidance](#)
[control allocation](#)
[LQR](#)
[MPC](#)
[gain scheduling](#)
[HIL / SITL](#)
[airworthiness](#)

Start here.

- PX4 developer docs, control architecture: docs.px4.io
- ArduPilot developer wiki: ardupilot.org/dev
- Beard & McLain, *Small Unmanned Aircraft: mavsim companion code*
- Brian Douglas, control system lectures (YouTube) for intuition on every loop we will tune
- acados, real-time MPC toolkit: [acados on GitHub](#)

Decide early.

- Fly on PX4/ArduPilot and study it, or write our own stack? (Suggest: fly on COTS, replace pieces as we prove ours.)
- Fixed-wing control structure: classical cascaded loops first; MPC/LQR as R&D once a validated model exists.
- How the plant model is obtained: from the test cell and nav team, or identified in flight?
- What “failsafe” means for the aid mission: return, loiter, or land in place?

2 Motor drive

Jayden Filleul • *needs members*

PURPOSE Turn DC from the pack into precisely timed three-phase power, efficiently, without letting the magic smoke out.

BLACK BOX *In:* bus voltage, throttle command, motor feedback. *Out:* three-phase drive, telemetry (current, RPM, temperature), faults.

MILESTONES **M0** requirements set; a COTS ESC characterised on the test cell as the baseline → **M1** six-step commutation spinning a motor on an evaluation board → **M2** own PCB, revision A → **M3** current control, protection, thermal validation → **M4** field-oriented control as R&D.

STACK KiCad or OrCAD; LTspice/PLECS for the power stage; STM32G4 or TI C2000 class MCU in C; SimpleFOC and VESC as open reference designs; oscilloscope with a current probe.

NEXT WEEK Write the one-page requirements sheet (voltage class, current, motor Kv range, protection); buy or borrow a motor-control evaluation kit; have the test cell measure a COTS ESC so there is a number to beat.

Self-directed research

[six-step](#)
[sensorless back-EMF](#)
[FOC / SVPWM](#)
[dead-time](#)
[gate driver](#)
[shunt vs inline sensing](#)
[DShot](#)
[regenerative braking](#)
[thermal derating](#)

Start here.

- VESC project, open hardware and firmware: vesc-project.com
- SimpleFOC, the gentlest FOC introduction: simplefoc.com
- AM32, open-source drone ESC firmware: github.com/am32-firmware/AM32
- ODrive documentation for current loop and calibration practice: docs.odriverobotics.com
- Vendor application notes: TI *sensorless FOC* and ST *Motor Control SDK* user manuals

Decide early.

- Build our own ESC or fly COTS and build ours as R&D? (Suggest: fly COTS for NFC, ship ours when the test cell says it is better.)
- Voltage class and current rating, driven by the pack decision from power systems.
- Sensorless six-step first, or straight to FOC with a position estimate?
- MCU family, which fixes the toolchain for the whole team.

3 Power systems

Changhan Zhang • Alfred Wang

PURPOSE Store the energy, deliver it safely on two independent buses, and always know how much is left.

BLACK BOX *In*: charged cells, load demand. *Out*: propulsion bus through the safety switch, regulated avionics rails, state of charge and pack health to the flight controller, protection trips.

MILESTONES **M0** cell chemistry and configuration chosen inside the rules; safety-switch design → **M1** pack built, charge and discharge characterised → **M2** BMS board: monitoring, balancing, protection → **M3** power distribution board and harness, dual-bus integration → **M4** state-of-charge estimation and telemetry to the flight controller.

STACK KiCad or OrCAD; LTspice; a BMS front-end evaluation board (TI BQ769x2 or ADI LTC68xx class); a logging charger; Python for cell data.

NEXT WEEK Extract every battery, switch and chemistry rule from the competition document into requirement IDs; shortlist cells and series/parallel configurations; sketch the two-bus topology with the safety switch placed.

Self-directed research

passive vs active balancing

coulomb counting

OCV-SoC

equivalent-circuit model

Kalman SoC

C-rate

internal resistance

BEC / regulation

XT60 / XT90

safety switch

Start here.

- Plett, *Battery Management Systems* vol. I–II, with the free companion course: [Coursera, Algorithms for BMS](#)
- foxBMS, open-source BMS platform: [foxbms.org](#)
- TI BQ76952 datasheet and evaluation module as a worked example of a monitor IC
- Competition rules, Appendix I battery voltage limits and requirements S002–S004

Decide early.

- Monitor IC plus microcontroller, or fully discrete? (Suggest: monitor IC.)
- Passive balancing is almost certainly enough; decide and move on.
- Is the power distribution board part of the BMS board or separate? Separate keeps the high-current path away from the sensing.
- Which state-of-charge method the flight controller will trust, and what it does when the pack says “low”.

4 Navigation & sensing

Lead TBD • Yifei Sun • Nicholas Tran

PURPOSE Turn noisy, biased sensors into a state estimate the controller can trust, and measure the airspeed and attitude the FDR must predict.

BLACK BOX *In*: IMU, barometer, GPS, magnetometer, airspeed. *Out*: attitude, angular rates, velocity, position, airspeed, wind estimate, health flags.

MILESTONES **M0** sensor selection and a test board or breakout → **M1** raw data logged; noise and Allan-variance characterisation → **M2** attitude filter (complementary or Mahony) validated on a bench rig → **M3** EKF fusing GPS, baro and mag → **M4** airspeed and wind estimation for the performance flights.

STACK Python (NumPy/SciPy) and MATLAB for design; C on the MCU; IMU evaluation boards (ICM-42688, BMI088 class); a rate table or a phone on a turntable for ground truth.

NEXT WEEK Shortlist three IMUs and one GPS; write the logging plan; implement a complementary filter on any IMU we already own and plot it against integrated gyro.

Self-directed research

Allan variance

complementary / Mahony / Madgwick

error-state EKF

quaternions

gyro bias

notch filtering

vibration isolation

GNSS

pitot / airspeed

Start here.

- Labbe, *Kalman and Bayesian Filters in Python*: [github.com/rlabbe](#)
- Solà, *Quaternion kinematics for the error-state Kalman filter*: [arXiv:1711.02508](#)
- PX4 EKF2 tuning guide, a production estimator explained: [docs.px4.io](#)
- Madgwick's original report and the Mahony filter paper; both are short

Decide early.

- Own sensor board, or use the sensors on the COTS autopilot and focus on the algorithms? (Suggest: algorithms first, board second.)
- Complementary filter for the first flights; EKF once there is ground truth to validate against.
- Where the estimator runs: on the autopilot, or on a separate processor talking over a bus?
- Ground-truth strategy: without one, no filter can be called validated.

5 Ground segment

Lead TBD • Wenrui Chen • Leah Chen

PURPOSE Everything that talks to the aircraft or learns from it after landing: the radio links, the ground station, logging, and the data pipeline that turns flights into FDR evidence.

BLACK BOX *In*: telemetry stream, onboard logs, mission plans. *Out*: live dashboard and alerts, mission upload, archived flight datasets, predicted-vs-measured plots.

MILESTONES **M0** radio link and MAVLink end to end on the bench → **M1** live telemetry dashboard → **M2** log pipeline: raw log in, standard plots out → **M3** mission upload and failsafe monitoring → **M4** FDR-ready analysis of every flight.

STACK Python (pymavlink, MAVSDK, pandas); QGroundControl or Mission Planner as the baseline GCS; PlotJuggler or Foxglove for live and replayed data; SiK/RFD900 or ExpressLRS radios; Git LFS for logs.

NEXT WEEK Get two telemetry radios talking with MAVLink heartbeats; define the flight log schema (what every flight must record); build a one-command script that turns a log into a first plot.

Self-directed research

MAVLink ULog 915 MHz ISM (AU) link budget ExpressLRS GCS / C2 time sync pandas
Grafana / Influx data provenance

Start here.

- MAVLink protocol and tooling: mavlink.io
- PX4 Flight Review, the reference for what a flight report looks like: review.px4.io
- PlotJuggler: plotjuggler.io • Foxglove: foxglove.dev
- ExpressLRS docs for the control link: expresslrs.org
- RFDesign (Australian) long-range telemetry modems: rfdesign.com.au

Decide early.

- Separate control and telemetry links, or one bidirectional link? (Suggest: separate; reliability over elegance.)
- Use QGroundControl and extend it, or write our own GCS in Python? (Suggest: QGC for flying, our own tooling for analysis.)
- Log format and the single schema every project must write to; this is the interface with everyone.
- Where the data lives: repository with LFS, or a shared drive with a manifest.

6 Propulsion test cell

Lead TBD • Zhen Rong (Drew) Fong

PURPOSE Measure, rather than guess, what every motor, propeller and ESC combination actually does. The FDR performance predictions start here.

BLACK BOX *In*: a motor, a prop, an ESC, a throttle profile. *Out*: thrust, torque, RPM, electrical power, efficiency and temperature versus throttle; a dataset in the ground segment's schema.

MILESTONES **M0** existing bench inventoried; measurands and accuracy targets defined → **M1** thrust, torque, RPM and electrical DAQ working and calibrated → **M2** first full motor and propeller map → **M3** automated throttle sweeps and ESC efficiency → **M4** dataset feeding the FDR propulsion model.

STACK Load cells with an ADC front end; RPM (optical or back-EMF); current and voltage sensing; an Arduino or STM32 DAQ; Python for acquisition and plotting; the Tyto Robotics stands as the commercial reference.

NEXT WEEK Photograph and list every part of the current bench; write the measurement list with target accuracy; run one manual test on any motor to find out what breaks.

Self-directed research

static thrust Kv / Kt propeller coefficients C_T , C_P advance ratio load-cell calibration DAQ sampling
uncertainty budget test procedure safety cage

Start here.

- UIUC propeller database, the standard for measured prop data: m-selig.ae.illinois.edu
- APC propeller performance data: apcprop.com
- Tyto Robotics thrust-stand documentation as a design reference: tytorobotics.com
- eCalc for quick sanity checks on motor and prop pairings: ecalc.ch

Decide early.

- Fix the existing bench or rebuild it around a proper load-cell arrangement?
- Static thrust only, or plan for a wind-tunnel or car-mounted dynamic test later?
- What the test procedure is, written down, before the first "real" data is taken.
- The data schema, agreed with the ground segment, so every dataset is comparable.

Cross-cutting

INTEGRATION AND REQUIREMENTS

Owned by the officers. One interface control document listing every connector, pinout, protocol and voltage; one requirements table mapping competition rules to IDs, projects and verification methods. Both live in the general `electronics` repository, and every project links to them rather than copying them.

WORKSHOPS, IN DEMAND ORDER

PID tuning and IMU filtering (9 of 10) • Git and GitHub (6) • Motors, ESCs and LiPo safety (6) • Low-level C (6) • Bench instruments (5) • KiCad (4) • Soldering (4) • Rules to requirements (3). The first three are the ones to run this month; they also cover exactly what the two under-staffed projects need.

REPOSITORY STRUCTURE

One general `electronics` repository for shared material, one repository per project with an identical skeleton (`README`, `hardware`, `firmware`, `sim`, `test`, `docs`), and a template repository for anything new. Existing repositories are renamed, not recreated.

CADENCE

One general meeting a week: progress, plan, problems, presented by the project leads. Design check-ins and boundary reviews between adjacent projects every few weeks. Rotation is expected and encouraged.

Decisions requested at this meeting

1. Confirm the six projects and the scope of each as written above.
2. Confirm the three leads (Rohan, Jayden, Changhan) and agree how the three vacant lead positions are filled: by nomination at the next general meeting once unallocated members have submitted the form.
3. Confirm officers own integration and requirements, and that the interface control document is started this week.
4. Approve the first three workshops and a date for the first one.
5. Approve the repository restructure so each project has a home before its first task is handed out.